Pre-promo QA gate for AI microdramas

Kuku AI Generalist assignment · Problem 3 · `python qa.py episode.mp4 --script episode.txt` → pass / review / fail + timestamped flags with frames

1. Problem and why it matters

Who: the promo editor, and the QA person before them, who cut clips from new AI-generated episodes every week. **Step:** just before an episode goes to promo cutting. **Output:** a verdict with timestamped flags for black frames, frozen picture, unexpected silence, distorted audio, outfit or background changes inside a scene, and broken lip sync. Nothing else. **Why it's real:** AI pipelines generate each shot separately and then stitch picture, voice and lip sync together, and those seams are where these defects appear. Trade coverage lists lip-sync mismatches as a common complaint about AI micro-dramas (Pandaily), and a 2026 short-drama generation paper names production QC that needs extensive manual review as unsolved (arXiv 2605.22144). My own test proved it: **2 of the 10 real Veo 3.1 clips I tested had lip-sync faults that nobody planted**, and the tool found both. **Why this over P1/P2:** a promo is paid reach. A frozen frame or out-of-sync line inside an ad burns spend at the moment a new viewer decides. This gate sits upstream of Problem 1, so every promo tool after it benefits.

2. Assumptions

• Inputs are the final episode file and the timestamped screenplay used to make it (scene headings with time ranges; `[mm:ss] NAME: line`). One parser function maps Kuku's real format.

• The screenplay defines *intent*. A scripted FADE or pause is not a defect. A scene is one location with a fixed cast and outfits unless an action line says otherwise. Without a script the tool still runs but loses most continuity accuracy.

• A missed blocker costs ad spend, while a false alarm costs seconds of editor time, so thresholds lean towards flagging and every flag is graded blocker / major / minor / info.

3. How it works: code decides, AI models measure, the editor confirms

Check	Method	Role of the script
Black / frozen	ffmpeg blackdetect, freezedetect (strict −72 dB: a talking head barely changes pixels)	scripted fade → info
Silence / distortion	ffmpeg silencedetect; clipped-sample share per 100 ms; loudness spikes; container A/V start offset	silence over a scripted line = blocker
Outfit / background	shot cuts → background colour with people masked; clothing colour under the main face (persisting, 1 flag/scene); optional vision-LLM second opinion (Gemini or any OpenAI-compatible, env-var swap; can only downgrade)	scene bounds, cast size, 'changes into' / 'walks into' lines
Lip sync	YuNet finds faces → MediaPipe Face Landmarker measures mouth opening on a zoomed face crop → cross-correlate with speech loudness ±0.5 s: consistent lag = offset; no relation = mismatch	which lines are on-screen (skips V.O.)

Verdict: any blocker → FAIL (regenerate), any major → REVIEW (the editor jumps to flagged timestamps, with frames in the HTML report), else PASS. Runs locally on a laptop CPU at about real time (an 8 s clip took ~10 s with face analysis), with no paid API.

4. How I know it's good

A. Real AI video (the main evidence). There's no public AI microdrama with a script and labelled faults, so I built 10 episodes from microsoft/AVGen-Bench (MIT): Veo 3.1, Sora 2, LTX-2 outputs with native speech. Each clip's generation prompt is its script. Technical faults were injected at known times, because published video has already been QC'd. Continuity breaks are *real*: two models' renders of the same prompt cut into one scene, which is what a regenerated shot looks like. Then I watched every clip to confirm flags.

Fault	Result
Black frames (injected 0.5 s) [r02]	caught (blocker)
Frozen picture (injected 1.5 s) [r03]	caught (blocker)
Silence over dialogue (injected 1.5 s) [r04]	caught (blocker)
Clipped audio (injected) [r05]	caught (major)
A/V offset 300 ms (injected) [r06]	caught (major)
Wrong audio track (injected) [r07]	MISSED: mouth could not be measured on these faces
Regenerated second half: other model, same prompt (real renders) [r08]	caught (major)
Regenerated 2 s insert shot (real renders) [r09]	caught (major)
Lip-sync mismatch in Veo output, found by the tool, confirmed by watching [r01 @0 s]	caught (minor)
Lip-sync mismatch in Veo output, found by the tool, confirmed by watching [r01 @16 s]	caught (major)

Totals: 9/10 faults flagged (8 as major/blocker), 9 false alarms (6 major/blocker): 4 background flags on trailer clips whose one prompt spans several places, 2 deliberate dips to black that the prompt-script doesn't mark, 1 outfit flag, 2 weak lip-sync flags (minor). **What the real clips changed:** MediaPipe alone found faces in 0–37% of frames (it's built for selfie-distance faces), so lip sync was never judged. YuNet + landmarker-on-crop raised that to 14–100% and caught the offset, but outfit false alarms jumped to 17. Judging clothing on the main face only, requiring persistence and one flag per scene cut total false alarms 26 → 11.

B. Synthetic seeded set (cartoon episodes; exact fault timing; thresholds tuned on the tuning set, then a held-out set built afterwards with smaller faults). With script: tuning 12/12 (12/12 major+), FA 0; held-out 10/10 (9/10 major+), FA 0. Without script: tuning 9/12 (7/12 major+), FA 22; held-out 8/10 (6/10 major+), FA 22. The script is what makes continuity work.

5. Where it fails

• **Wrong audio track (r07) missed:** faces were on screen, but the landmarker couldn't measure the mouth at those angles, so lip sync was skipped, not failed. Mouth measurement on profile or small faces is the weakest link.

• **Lip sync** correlates loudness with mouth opening. It can't catch the wrong words being mouthed, and it's a weak signal on real footage, so mismatches are minor unless there's no relation at all.

• **Intent comes from the script.** Unscripted stylistic fades get flagged, and multi-location scenes not marked in the script get background flags. A real screenplay fixes most of this; a prompt doesn't.

• **Outfit = colour.** A same-colour swap or a missing accessory is missed. Clips are 8–12 s, not 1–2 min. 10 real episodes is a small sample: next step is ~20 real Kuku episodes labelled by an editor.

6. Scale, cost, and how a real team would know it worked

All pass/fail logic is local ffmpeg / OpenCV / MediaPipe: roughly 2–3 CPU-minutes per 2-min episode (measured ~1.2× real time), run in parallel per episode on any worker queue at near-zero cost. The optional LLM sees only flagged frame pairs (0–3 calls per episode). **Pilot metrics:** share of flags the editor marks real (target >60%); blocker-level defects later found in published promos (target ≈0); editor minutes per episode on technical review; regenerations moved before promo cutting. Every editor decision is kept as a label to re-tune thresholds per show style.

7. AI tools used, and where they got it wrong

Built with Claude (planning, code, test sets, this doc). I reviewed every decision and threshold and ran everything on my machine. Where the AI got it wrong, each found by running the tool and reading its output: ffmpeg's default freeze threshold called a clean talking head 'frozen' 15 times; its first loudness-spike detector flagged almost every syllable; it wrote MediaPipe code for an API that no longer ships; its scoring script counted catches as false alarms; a Windows file-encoding 'fix' broke the test generator; it analysed frames at 360 px wide, too small for faces in landscape clips; and it assumed MediaPipe would find cinematic faces, which the real clips disproved.